

Analítica Espacial: Arreglos y Tablas

Santiago Gallego

2026-05-07

Contenido

0.1	Ejercicio 1: Vectorización y álgebra espacial (Haversine)	2
0.1.1	python	2
0.1.2	R	3
0.1.3	python	3
0.1.4	R	3
0.1.5	python	4
0.1.6	R	4
0.2	Ejercicio 2: Limpieza, filtros y agregación tabular	5
0.2.1	python	5
0.2.2	R	5
0.2.3	python	5
0.2.4	R	6
0.2.5	python	6
0.2.6	R	6
0.2.7	python	7
0.2.8	R	7

0.1 Ejercicio 1: Vectorización y álgebra espacial (Haversine)

La fórmula de Haversine permite calcular la gran distancia sobre la esfera terrestre dadas dos coordenadas en latitud y longitud.

Tu misión es crear una función vectorizada (en Python, R o Julia) llamada `calcular_distancias` que reciba como argumento el DataFrame `df` de censos (que construimos en la sección 4) y calcule la distancia en kilómetros desde Bogotá (Lat: 4.7110, Lon: -74.0721) a todas las demás ciudades de la tabla, agregando el resultado en una nueva columna llamada `Distancia_Bogota`.

0.1.1 python

```
import pandas as pd
import numpy as np

# 1. Datos iniciales
datos_censo = {
    "Ciudad": ["Bogotá", "Medellín", "Cali", "Barranquilla"],
    "Latitud": [4.7110, 6.2476, 3.4516, 10.9639],
    "Longitud": [-74.0721, -75.5658, -76.5320, -74.7964]
}
```

```
# Convertimos el diccionario a DataFrame para aprovechar el procesamiento en bloque
df = pd.DataFrame(datos_censo)
```

0.1.2 R

```
# 1. Datos iniciales (incluyendo coordenadas)
# Usamos un data.frame para manejar los datos de forma tabular y facilitar cálculos vectoriales
df <- data.frame(
  Ciudad = c("Bogotá", "Medellín", "Cali", "Barranquilla"),
  Latitud = c(4.7110, 6.2476, 3.4516, 10.9639),
  Longitud = c(-74.0721, -75.5658, -76.5320, -74.7964)
)
```

0.1.3 python

```
# 2. Definición de la función vectorizada
def calcular_distancias(dataframe):
    # Coordenadas de Bogotá (Punto de referencia fijo)
    lat_bog = 4.7110
    lon_bog = -74.0721
    R = 6371.0 # Radio medio de la Tierra en km

    # np.radians es ideal aquí porque procesa toda la columna (Series) a la vez sin ciclos for
    lat1, lon1 = np.radians(lat_bog), np.radians(lon_bog)
    lat2 = np.radians(dataframe['Latitud'])
    lon2 = np.radians(dataframe['Longitud'])

    # Al restar columnas completas, NumPy aplica álgebra lineal optimizada
    dlat = lat2 - lat1
    dlon = lon2 - lon1

    # Aplicamos la fórmula de Haversine usando funciones de NumPy para mantener la vectorización
    a = np.sin(dlat / 2)**2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2)**2
    c = 2 * np.arcsin(np.sqrt(a))

    # Asignamos el resultado a una nueva columna directamente
    dataframe['Distancia_Bogota'] = R * c
    return dataframe
```

0.1.4 R

```
# 2. Definición de la función vectorizada
calcular_distancias <- function(dataframe) {
```

```

lat_bog <- 4.7110
lon_bog <- -74.0721
R <- 6371

# En R, la multiplicación por pi/180 sobre un vector es intrínsecamente vectorizada
lat1 <- lat_bog * pi / 180
lon1 <- lon_bog * pi / 180
lat2 <- dataframe$Latitud * pi / 180
lon2 <- dataframe$Longitud * pi / 180

# Las diferencias se calculan entre vectores del mismo tamaño
dlat <- lat2 - lat1
dlon <- lon2 - lon1

# Usamos las funciones matemáticas base de R que están optimizadas para vectores
a <- sin(dlat / 2)^2 + cos(lat1) * cos(lat2) * sin(dlon / 2)^2
c <- 2 * asin(sqrt(a))

# Creamos la columna usando el operador $ para modificar el data.frame original
dataframe$Distancia_Bogota <- R * c

return(dataframe)
}

```

0.1.5 python

```

# 3. Ejecución
df_final = calcular_distancias(df)
print(df_final[['Ciudad', 'Distancia_Bogota']])

```

	Ciudad	Distancia_Bogota
0	Bogotá	0.000000
1	Medellín	237.754393
2	Cali	306.670166
3	Barranquilla	699.848833

0.1.6 R

```

# 3. Ejecución y visualización
df_final <- calcular_distancias(df)
print(df_final[, c("Ciudad", "Distancia_Bogota")])

```

	Ciudad	Distancia_Bogota
1	Bogotá	0.0000

2	Medellín	237.7544
3	Cali	306.6702
4	Barranquilla	699.8488

0.2 Ejercicio 2: Limpieza, filtros y agregación tabular

Para consolidar tus habilidades analíticas con DataFrames, crea un script que resuelva el siguiente flujo de trabajo simulado:

1. Creación: Genera un DataFrame con los siguientes datos de estaciones meteorológicas: Estación: ["Páramo", "Valle", "Costa", "Selva", "Sabana"] Elevación: [3200, 1500, 15, 200, 2600] Precipitación: [850.5, 1200.0, None, 3000.5, None] (Usa el tipo de nulo apropiado según tu lenguaje: None/np.nan, NA o missing).

0.2.1 python

```
import pandas as pd
import numpy as np

# 1. Creación con nulos (NaN): Usamos np.nan porque Pandas lo reconoce como nulo numérico
datos = {
    "Estacion": ["Páramo", "Valle", "Costa", "Selva", "Sabana"],
    "Elevacion": [3200, 1500, 15, 200, 2600],
    "Precipitacion": [850.5, 1200.0, np.nan, 3000.5, np.nan]
}
df = pd.DataFrame(datos)
```

0.2.2 R

```
# 1. Creación con nulos (NA): NA es el valor lógico estándar de R para datos faltantes
df <- data.frame(
  Estacion = c("Páramo", "Valle", "Costa", "Selva", "Sabana"),
  Elevacion = c(3200, 1500, 15, 200, 2600),
  Precipitacion = c(850.5, 1200.0, NA, 3000.5, NA),
  stringsAsFactors = FALSE
)
```

2. Imputación: Los sensores de la Costa y la Sabana fallaron. Rellena los datos de precipitación nulos utilizando el valor promedio (media) de las estaciones válidas.

0.2.3 python

```
# 2. Imputación: Usamos .mean() porque ignora automáticamente los nulos para el cálculo
media_precip = df["Precipitacion"].mean()

# .fillna() es la función estándar de Pandas para reemplazar nulos sin borrar filas completas
df["Precipitacion"] = df["Precipitacion"].fillna(media_precip)
```

0.2.4 R

```
# 2. Imputación
# 2. Imputación: Usamos na.rm = TRUE para que R no devuelva NA como resultado del promedio
media_precip <- mean(df$Precipitacion, na.rm = TRUE)

# is.na() es la función correcta para indexar celdas vacías y asignarles un nuevo valor
df$Precipitacion[is.na(df$Precipitacion)] <- media_precip
```

- La Imputación (Manejo de Nulos) Rellenar con la media es una técnica de Imputación Simple. Es vital porque la mayoría de los algoritmos de Machine Learning o modelos espaciales fallan si encuentran un valor NaN/NA.

En Python: fillna() es un método muy potente que permite incluso rellenar con el valor anterior o siguiente.

En R: is.na() es un filtro lógico que identifica las celdas vacías para poder actuar sobre ellas.

3. Filtro espacial (Slicing): Extrae en un nuevo DataFrame llamado estaciones__altas solo aquellas filas cuya elevación sea estrictamente mayor a 1000 metros.

0.2.5 python

```
# 3. Filtro espacial: Slicing por condición
# Usamos una máscara booleana (df["Elevacion"] > 1000)
# .copy() asegura que 'estaciones_altas' sea un nuevo objeto y no una vista del anterior
estaciones_altas = df[df["Elevacion"] > 1000].copy()
```

0.2.6 R

```
# 3. Filtro: Usamos el sistema de coordenadas [filas, columnas] de R
# La condición df$Elevacion > 1000 selecciona solo las filas que cumplen la regla
estaciones_altas <- df[df$Elevacion > 1000, ]
```

- Slicing y Filtrado Al filtrar por Elevacion > 1000, estamos reduciendo el “universo” de nuestro análisis.

Importante: En Python, usamos `.copy()` al crear `estaciones_altas`. Esto evita el famoso `SettingWithCopyWarning`, asegurando que el nuevo `DataFrame` sea independiente del original en la memoria.

4. Cálculo final: Muestra en consola cuál es la precipitación promedio anual reportada exclusivamente en las estaciones altas.

0.2.7 python

```
# 4. Cálculo final
promedio_altas = estaciones_altas["Precipitacion"].mean()

print("--- REPORTE PYTHON ---")
```

```
--- REPORTE PYTHON ---
```

```
print(f"Media de imputación: {media_precip:.2f}")
```

```
Media de imputación: 1683.67
```

```
print(estaciones_altas)
```

	Estacion	Elevacion	Precipitacion
0	Páramo	3200	850.500000
1	Valle	1500	1200.000000
4	Sabana	2600	1683.666667

```
print(f"\nPrecipitación promedio en zonas altas: {promedio_altas:.2f} mm")
```

```
Precipitación promedio en zonas altas: 1244.72 mm
```

0.2.8 R

```
# 4. Cálculo final: Aquí ya no necesitamos na.rm porque ya imputamos los datos
promedio_altas <- mean(estaciones_altas$Precipitacion)

cat("--- REPORTE R ---\n")
```

```
--- REPORTE R ---
```

```
cat(sprintf("Media de imputación: %.2f\n", media_precip))
```

Media de imputación: 1683.67

```
print(estaciones_altas)
```

	Estacion	Elevacion	Precipitacion
1	Páramo	3200	850.500
2	Valle	1500	1200.000
5	Sabana	2600	1683.667

```
cat(sprintf("\nPrecipitación promedio en zonas altas: %.2f mm\n", promedio_altas))
```

Precipitación promedio en zonas altas: 1244.72 mm

- Agregación Selectiva El cálculo final demuestra cómo una estadística descriptiva (el promedio) cambia drásticamente cuando se aplica a un subconjunto específico de datos en lugar de a toda la tabla.